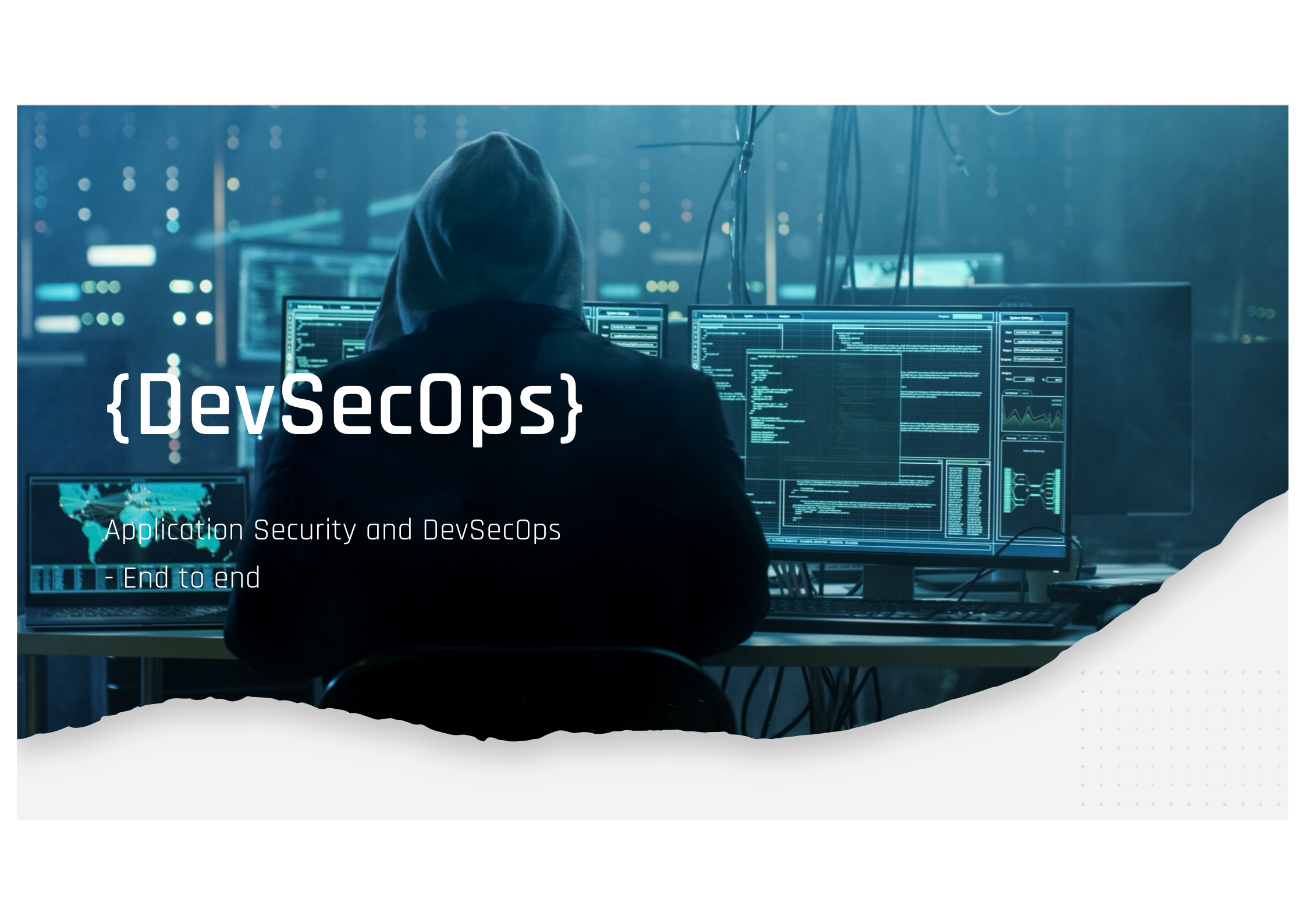
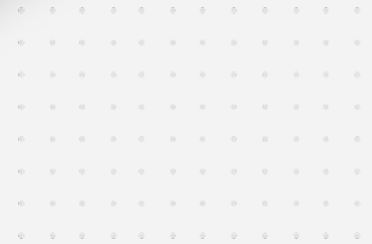


A person wearing a dark hoodie is seen from behind, sitting at a desk in a server room. They are looking at several computer monitors. The monitors display various technical interfaces, including code editors, system logs, and network maps. The room is dimly lit with blue ambient light from the screens and server racks in the background. The overall atmosphere is one of focused, technical work.

{DevSecOps}

Application Security and DevSecOps
- End to end



Losses caused by **cyber attacks** reported to IC3

2021

\$ 6.900.000.000

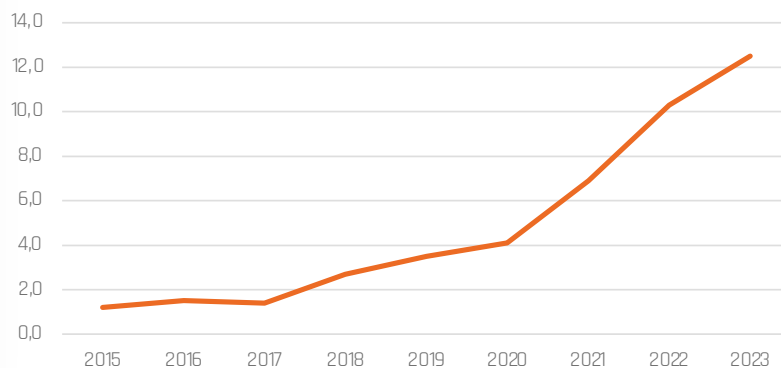
2023

\$ 12.500.000.000



Loss caused
by reported
cyber crime
(in billion
USD)

Loss caused by reported cyber crime
(in billion USD)



Top 5 crime types:

- > Phishing
- > Non-Payment / Delivery
- > Data Breach
- > Identity Theft
- > Extortion



Trends

- > Confidence fraud / Romance scams
- > Cryptocurrency
- > Ransomware
- > Tech support fraud



Image thanks to Gregor Biswanger @BFreakout



Hackers in movies
vs

© 2021 Michael Kaufmann @mike_kaufmann



How people think they get hacked



How they really get hacked

If you had to marry your spouse where you met them, where would your wedding have been? 14.5K · 100K Comments · 2.3K Shares	Your porn name, is your middle name, and the first car you had. 1.2K · 26.4K Comments · 9.27 Shares	How far away do you live from the place you were born? 1.3K · 10.9K Comments · 2.8K Shares
The car you passed your drivers test in was a _____ 14.5K · 100K Comments · 8.9K Shares	STOP. THINK. DO NOT SHARE INFO.	NAME A SONG THAT TAKES YOU BACK TO HIGH SCHOOL

Michael Kaufmann

Founder & Managing
Director, Xebia Germany



@mike_kaufmann



@wulfland

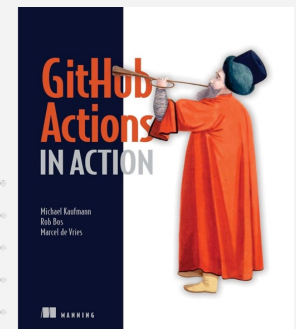


<https://writeabout.net>



Microsoft®
Most Valuable
Professional

- > 25 years software developer
- > 15 years ALM & DevOps
- > 10 years Git and GitHub
- > Microsoft RD & MVP



The event-stream incident



Social engineering attack



Supply chain attack:
event-stream@3.3.6 -> flatmap-stream@0.1.1



Code execution in build process
targeting **copay**



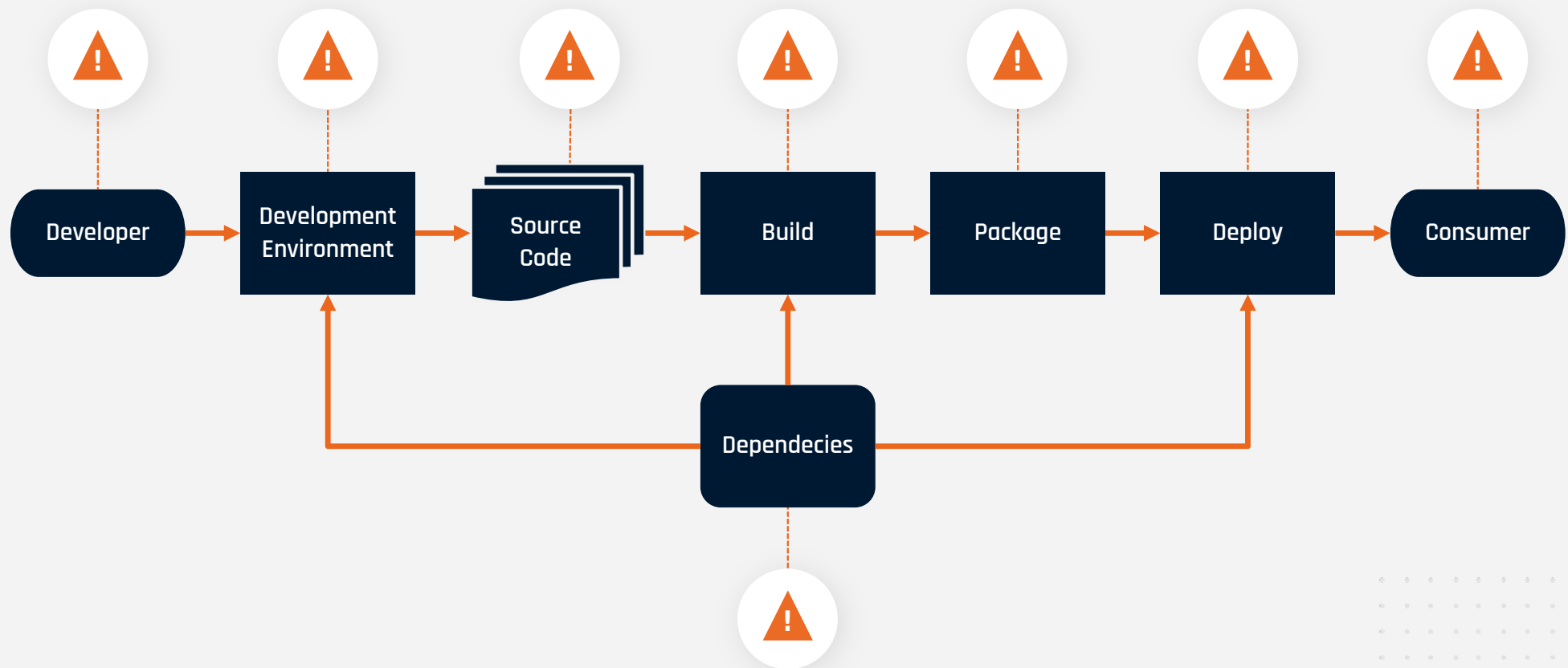
Harvest the user's bitcoin and
private keys

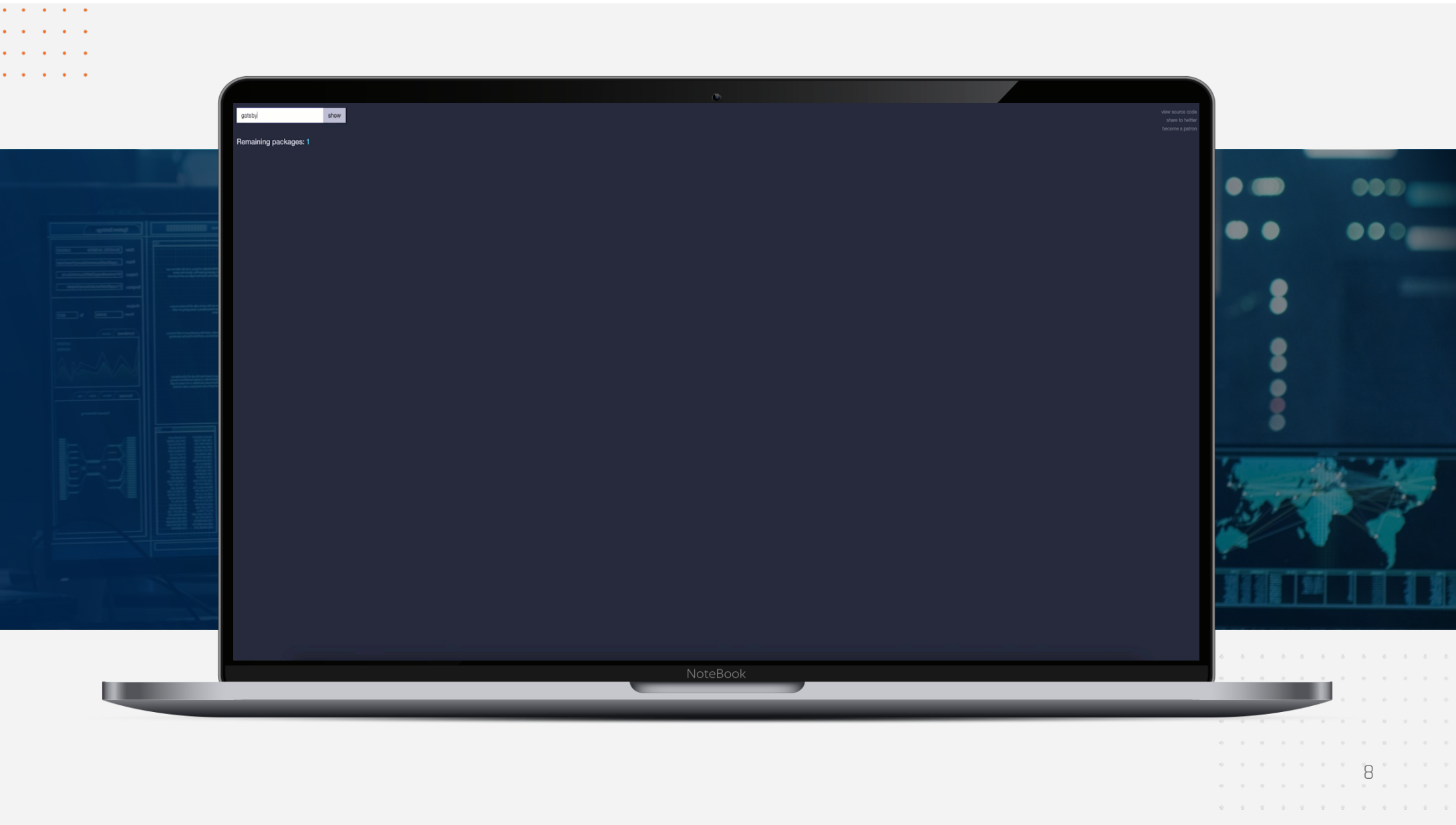
The screenshot shows a GitHub Security Advisory titled "Malicious Package in flatmap-stream". The advisory is marked as "Critical severity" and was published on September 1, 2020, with an update on October 1, 2021. It details a vulnerability in the `flatmap-stream` npm package, specifically version 0.1.1. The description states that this module runs an encrypted payload targeting a very specific application, `copay`, and because they shared the same description it would have likely worked for `copay-dash`. The advisory lists the injected code, which includes reading AES encrypted data from a file disguised as a test fixture, grabbing the npm package description of the module that imported it, and using the package description as a key to decrypt a chunk of data pulled in from the disguised file. It also lists the actions performed by the module, such as decrypting another chunk of data, concatenating a small, commented prefix, performing minor decoding tasks, and writing the processed block of JS out to a file stored in a dependency that would be packaged by the build scripts. The advisory concludes that the chunk of code that was written out was the actual malicious code, intended to be run on devices owned by the end users of Copay. A section titled "This code would do the following:" is highlighted with an orange box, listing the following actions:

- Detect the current environment: Mobile/Cordova/Electron
- Check the Bitcoin and Bitcoin Cash balances on the victim's copay account
- If the current balance was greater than 100 Bitcoin, or 1000 Bitcoin Cash:
 - Harvest the victim's account data in full
 - Harvest the victim's copay private keys
 - Send the victim's account data/private keys off to a collection

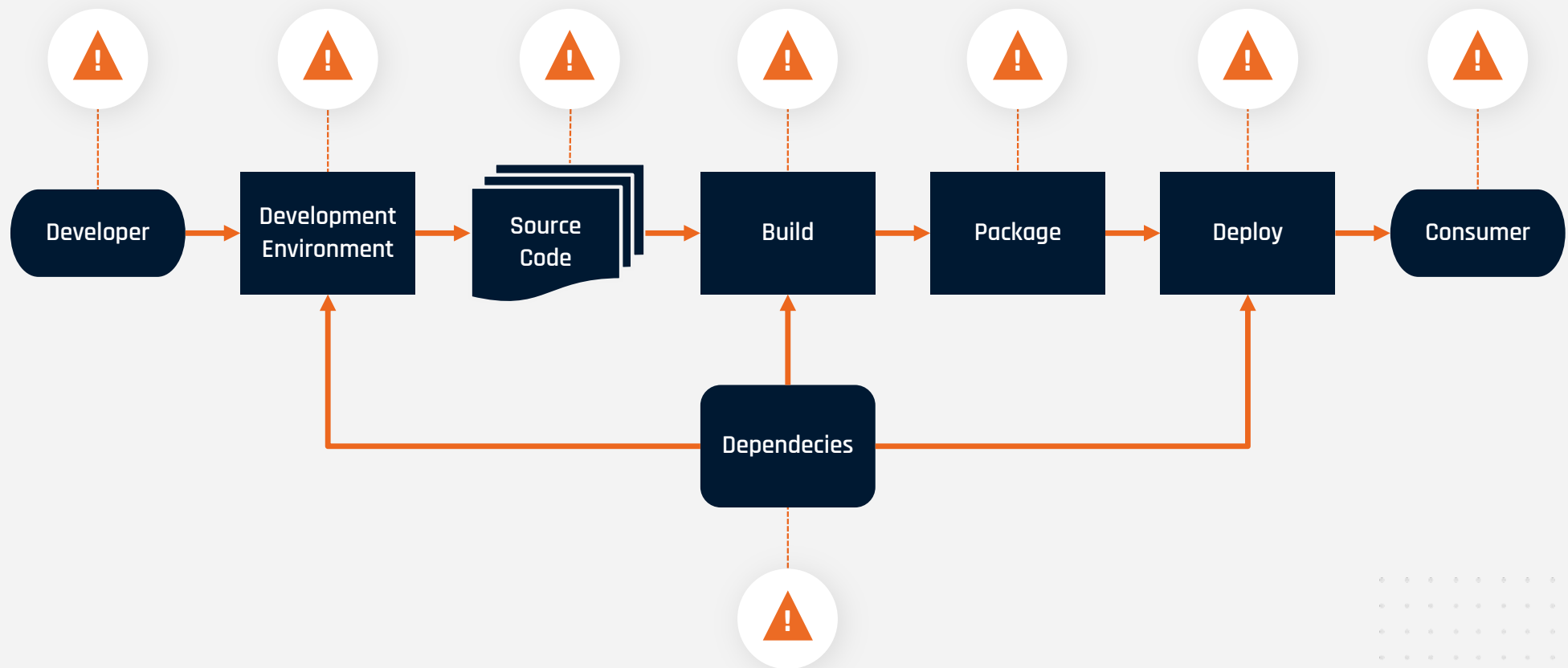
On the right side of the screenshot, the user profile for `right9ctrl` is visible, showing their location as Tokyo and a "Follow" button. The GHSA ID is GHSA-9x64-5r7x-2q53, and the CVSS Score is 9.8 Critical.

Attack vectors

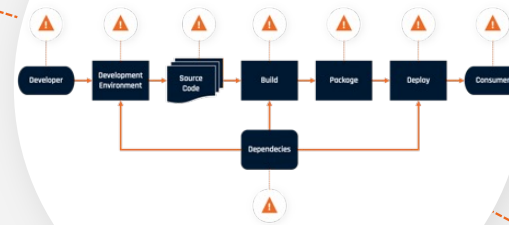
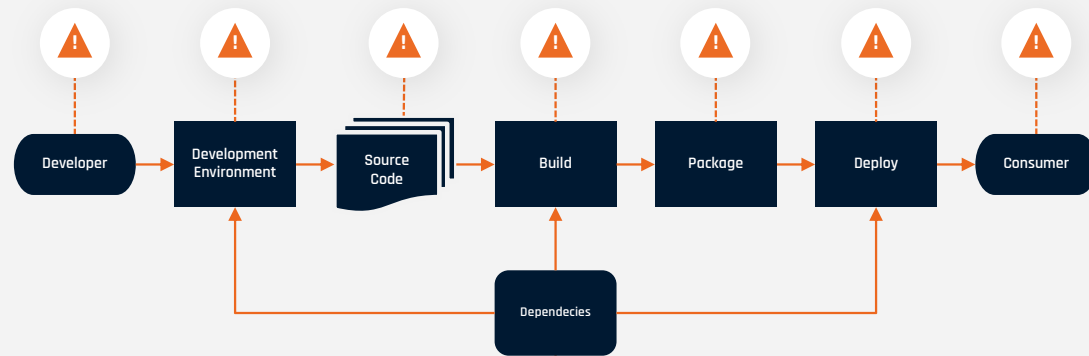




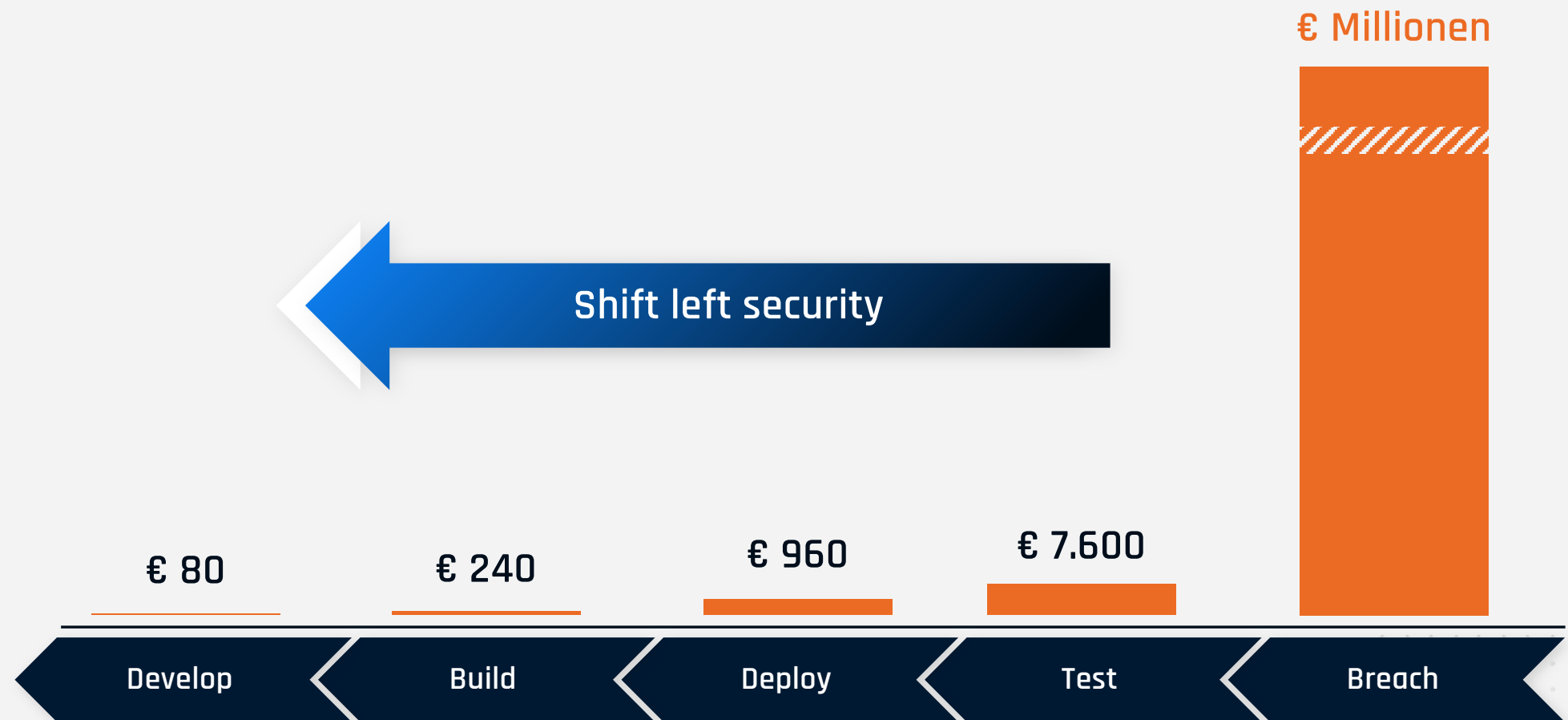
Attack vectors



Attack vectors



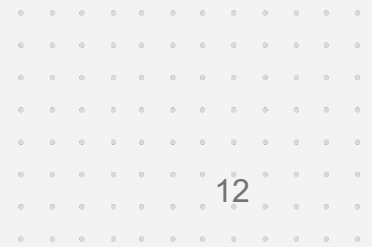
Costs for fixing a security vulnerability



Source: Ponemon Institute Cost of a Data Breach 2020



Supply Chain Security



Software Composition Analysis (SCA)



GitHub (Dependency-Graph/Dependabot)



anchore (<https://anchore.com/>)



Dependency-Track
(<https://dependencytrack.org/>)

Dependabot alerts

4 Open ✓ 0 Closed		Manifest ▾	Sort ▾
🚩	marsdb	3 minutes ago by GitHub package.json	critical severity
🚩	express-jwt	3 minutes ago by GitHub package.json	high severity
🚩	sanitize-html	3 minutes ago by GitHub package.json	moderate severity
🚩	jsonwebtoken	3 minutes ago by GitHub package.json	critical severity

Dependency graph

Dependencies Dependents Dependabot

⚠️ We found potential security vulnerabilities in your dependencies.
Dependencies defined in these manifest files have known security vulnerabilities and should be updated:

package.json 7 vulnerabilities found

[View Dependabot alerts](#)

Only the owner of this repository can see this message.

These dependencies are defined in workshop-2021-learning-journey's manifest files, such as [package.json](#) and [frontend/package.json](#).

📦 Dependencies defined in package.json 138

>	auth0 / express-jwt	Known security vulnerability in 0.1.3 ▾
>	auth0 / node-jsonwebtoken jsonwebtoken	Known security vulnerability in 0.4.0 ▾
>	c58 / marsdb	Known security vulnerability in ^ 0.6.11 ▾
>	apostrophecms / sanitize-html	Known security vulnerability in 1.4.2 ▾
>	istanbuljs / istanbuljs @istanbuljs/nyc-config-typescript	^ 1.0.1
	Seally / types-chai @types/chai	^ 4.2.14
>	DT DefinitelyTyped / DefinitelyTyped @types/chai-as-promised	^ 7.1.3

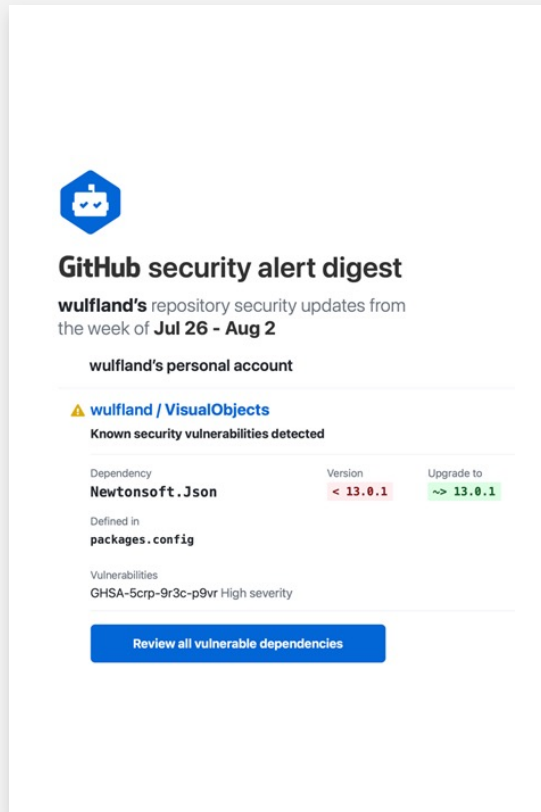
Dependency Management

GitHub Dependency graph

▶ Dependabot alerts

▶ Dependabot security updates

▶ Dependabot version updates



GitHub security alert digest
wulfland's repository security updates from the week of **Jul 26 - Aug 2**

wulfland's personal account

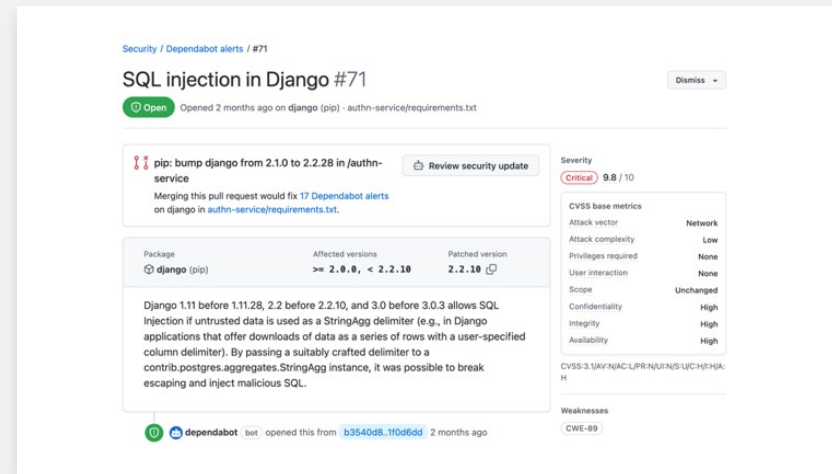
⚠ wulfland / VisualObjects
Known security vulnerabilities detected

Dependency	Version	Upgrade to
Newtonsoft.Json	< 13.0.1	~> 13.0.1

Defined in packages.config

Vulnerabilities
GHSA-5crp-9r3c-p9vr High severity

[Review all vulnerable dependencies](#)



Security / Dependabot alerts / #71

SQL injection in Django #71

[Open](#) Opened 2 months ago on django (pip) · authn-service/requirements.txt

🔴 pip: bump django from 2.1.0 to 2.2.28 in /authn-service

Merging this pull request would fix 17 Dependabot alerts on django in authn-service/requirements.txt.

[Review security update](#)

Package	Affected versions	Patched version
django (pip)	>= 2.0.0, < 2.2.10	2.2.10

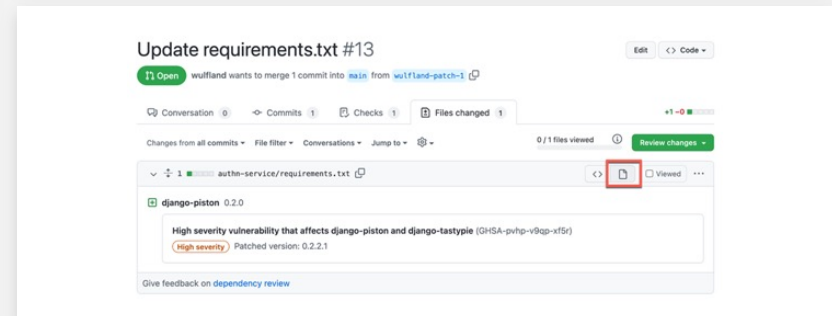
Django 1.11 before 1.11.28, 2.2 before 2.2.10, and 3.0 before 3.0.3 allows SQL injection if untrusted data is used as a StringAgg delimiter (e.g., in Django applications that offer downloads of data as a series of rows with a user-specified column delimiter). By passing a suitably crafted delimiter to a contrib.postgres.aggregates.StringAgg instance, it was possible to break escaping and inject malicious SQL.

Severity: **Critical** 9.8 / 10

CVSS base metrics	Network
Attack vector	Low
Attack complexity	None
Privileges required	None
User interaction	None
Scope	Unchanged
Confidentiality	High
Integrity	High
Availability	High

Weaknesses
CWE-89

[dependabot](#) bot opened this from b354098.1f0d6dd 2 months ago



Update requirements.txt #13

[Open](#) wulfland wants to merge 1 commit into main from wulfland-patch-1

Conversation · Commits · Checks · Files changed 1

Changes from all commits · File filter · Conversations · Jump to · 0 / 1 files viewed · [Review changes](#)

authn-service/requirements.txt

django-piston 0.2.0

High severity vulnerability that affects django-piston and django-tastypie (GHSA-pvhp-v9qp-xf5r)
High severity Patched version: 0.2.2.1

[Give feedback on dependency review](#)

Dependency Management

GitHub Dependency graph

▶ Dependabot **alerts**

▶ Dependabot
security updates

▶ Dependabot
version updates

```
jobs:
  dependabot:
    runs-on: ubuntu-latest
    if: ${{ github.actor == 'dependabot[bot]' }}
    steps:
      - name: Dependabot metadata
        id: dependabot-metadata
        uses: dependabot/fetch-metadata@v1.1.1
        with:
          github-token: "${{ secrets.GITHUB_TOKEN }}"
      - name: Enable auto-merge for all patch versions
        if: ${{ steps.metadata.outputs.update-type == 'version-update:semver-patch' }}
        run: gh pr merge --auto --merge "$PR_URL"
        env:
          PR_URL: ${{ github.event.pull_request.html_url }}
          GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}
```

Bump terser from 4.8.0 to 4.8.1 in /frontend #50

[Open](#) dependabot wants to merge 1 commit into `main` from `dependabot/npm_and_yarn/frontend/terser-4.8.1`

Conversation 0 · Commits 1 · Checks 6 · Files changed 1 · +14 -22

dependabot bot · commented 14 days ago

Bumps `terser` from 4.8.0 to 4.8.1.

► Changelog

► Commits

compatibility 97%

Dependabot will resolve any conflicts with this PR as long as you don't alter it yourself. You can also trigger a rebase manually by commenting `@dependabot rebase`.

► Dependabot commands and options

dependabot bot · added `dependencies` `javascript` labels 14 days ago

Reviewers

No reviews

Still in progress? Convert to draft

Assignees

No one—assign yourself

Labels

`dependencies` `javascript`

Projects

None yet

Milestone

No milestone

Development

Successfully merging this pull request may



Artifact attestations to establish provenance for builds

[actions/attest-sbom: Action for generating SBOM attestations for workflow artifacts \(github.com\)](#)

[actions/attest: Action for generating attestations for workflow artifacts \(github.com\)](#)

[Using artifact attestations to establish provenance for builds - GitHub Docs](#)

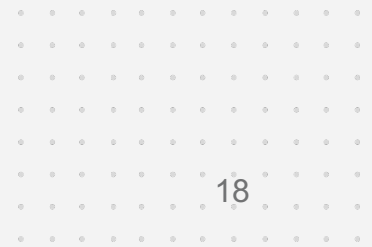
The SLSA framework is an industry standard used to evaluate supply chain security. It is organized into levels. Each level represents an increasing degree of security and trustworthiness for a software supply chain. Artifact attestations provides SLSA v1.0 Build Level 2.



Demo: Demo: Dependabot



Developer Security





Attacking developers

Phishing / Spear Phishing

Social engineering

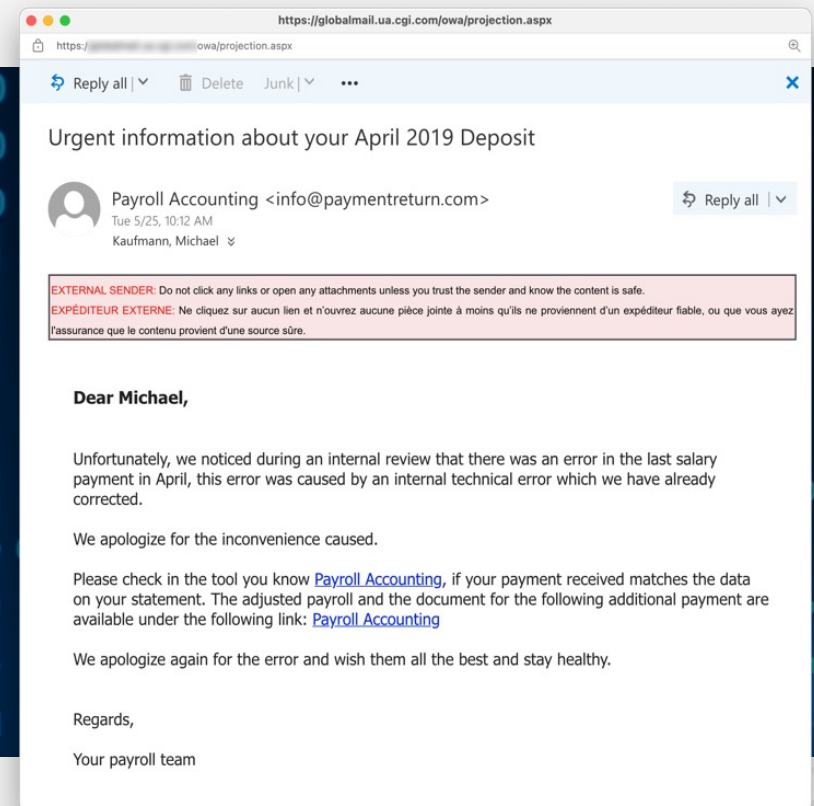
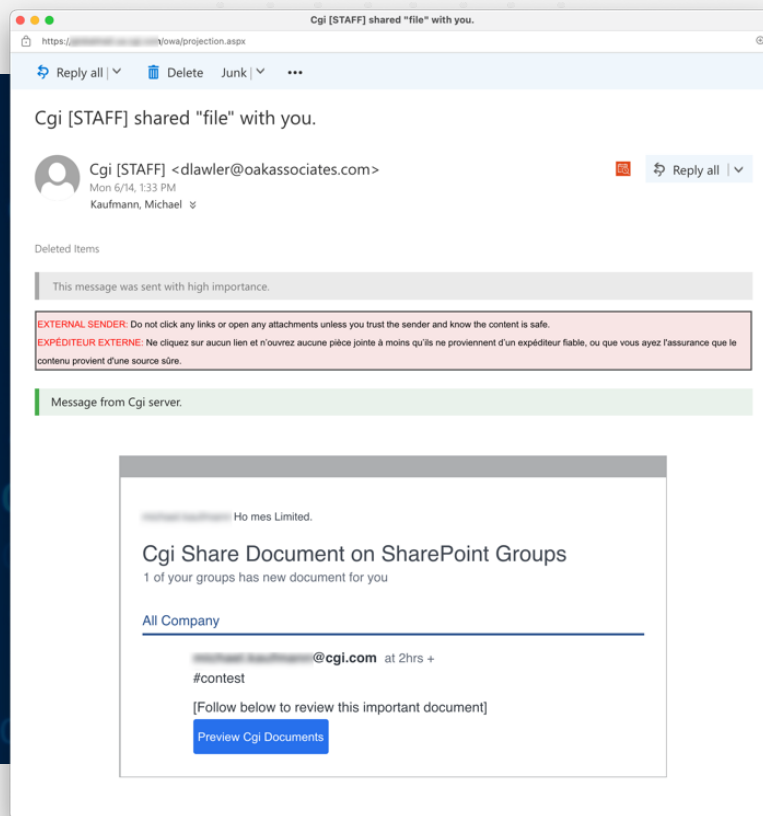
Unsecured connections to test systems



“A developer is just a normal employee – that works as local admin, can push and execute code on various systems in minutes, and often runs unsecured web servers.”



Phishing



Attacking **developers**

Typo squatting



Namespace shadowing



Typo squatting

```
$ npm install crossenv
```



Steals all
your
environment
variables

```
$ npm install cross-env
```



Normal
package



Namespace shadowing

```
$ npm install @azure/core-tracing
```



Normal
package

```
$ npm install core-tracing
```



Upload data to
a control server



Credentials **Developer**

E-Mail	→ Spear phishing
Access machines	→ Log on, Mimikatz
Source	→ Inject code
Pipeline	→ Execute code / scripts
Access test environment	→ Test against prod?
Access prod?	



Virtual DevEnv

- ▶ Virtual development environments

- ▶ Specific for project

- ▶ No local admin rights

- ▶ Codespaces / DevBox

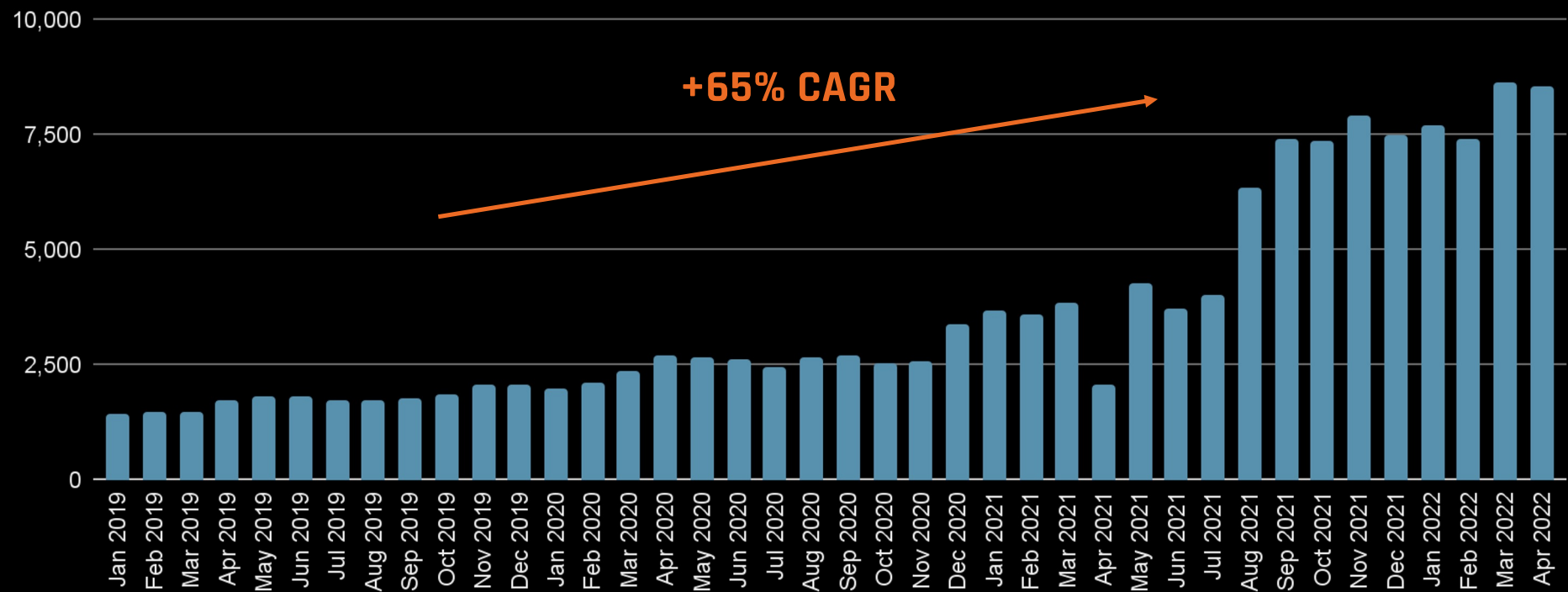
- ▶ Secret scanning



Secret Scanning

We're seeing more credential leaks than ever

GitHub access tokens leaked in public repositories



Secret Scanning

▶ Code

- › GitHub Secret Scanning
- › gitLeaks
- › SpectralOps
- › Git-Secrets
- › Whispers
- › Gittyleaks
- › Git-all-secrets
- › ...

▶ Fileshare

- › Bash/PowerShell
- › Dumpster

Adafruit IO Adafruit IO Key	Dropbox Dropbox Access Token Dropbox Short Lived Access Token	Plivo Plivo Auth Token
Adobe Adobe Device Token Adobe JSON Web Token Adobe Service Token Adobe Short-Lived Access Token	Dynatrace Dynatrace Access Token Dynatrace Internal Token	Postman Postman API Key
Alibaba Cloud Alibaba Cloud Access Key ID and Access Key Secret pair	Finicity Finicity App Key	Proctorio Proctorio Consumer Key Proctorio Linkage Key Proctorio Registration Key Proctorio Secret Key
Amazon Web Services (AWS) Amazon AWS Access Key ID and Secret Access Key pair	Frame.io Frame.io Developer Token Frame.io JSON Web Token	Pulumi Pulumi Access Token
Atlassian Atlassian API Token Atlassian JSON Web Token	GitHub GitHub App Installation Access Token GitHub OAuth Access Token GitHub Personal Access Token GitHub Refresh Token GitHub SSH Private Key	PyPI PyPI API Token
Azure Azure Active Directory Application Secret Azure DevOps Personal Access Token Azure SAS Token Azure Service Management Certificate Azure SQL Connection String Azure Storage Account Key	GoCardless GoCardless Live Access Token GoCardless Sandbox Access Token	RubyGems RubyGems API Key
Clojars Clojars Deploy Token	Google Cloud Google API Key Google Cloud Private Key ID	Samsara Samsara API Token Samsara OAuth Access Token
CloudBees CodeShip CloudBees CodeShip Credential	Hashicorp Terraform Terraform Cloud / Enterprise API Token	SendGrid SendGrid API Key
Databricks Databricks Access Token	Hubspot Hubspot API Key	Shopify Shopify Access Token Shopify App Shared Secret Shopify Custom App Access Token Shopify Private App Password
Datadog Datadog API Key	Mailchimp Mailchimp API Key Mandrill API Key	Slack Slack API Token Slack Incoming Webhook URL Slack Workflow Webhook URL
Discord Discord Bot Token	Mailgun Mailgun API Key	SSLMate SSLMate API Key SSLMate Cluster Secret
Doppler Doppler CLI Token Doppler Personal Token Doppler SCIM Token Doppler Service Token	MessageBird MessageBird API Key	Stripe Stripe Live API Restricted Key Stripe Live API Secret Key Stripe Test API Restricted Key Stripe Test API Secret Key
	npm npm Access Token	Tencent Cloud Tencent Cloud Secret ID
	NuGet NuGet API Key	Twilio Twilio Account String Identifier Twilio API Key
	OpenAI OpenAI API Key	Valour Valour Access Token
	Palantir Palantir JSON Web Token	



Demo: Secret Scanning

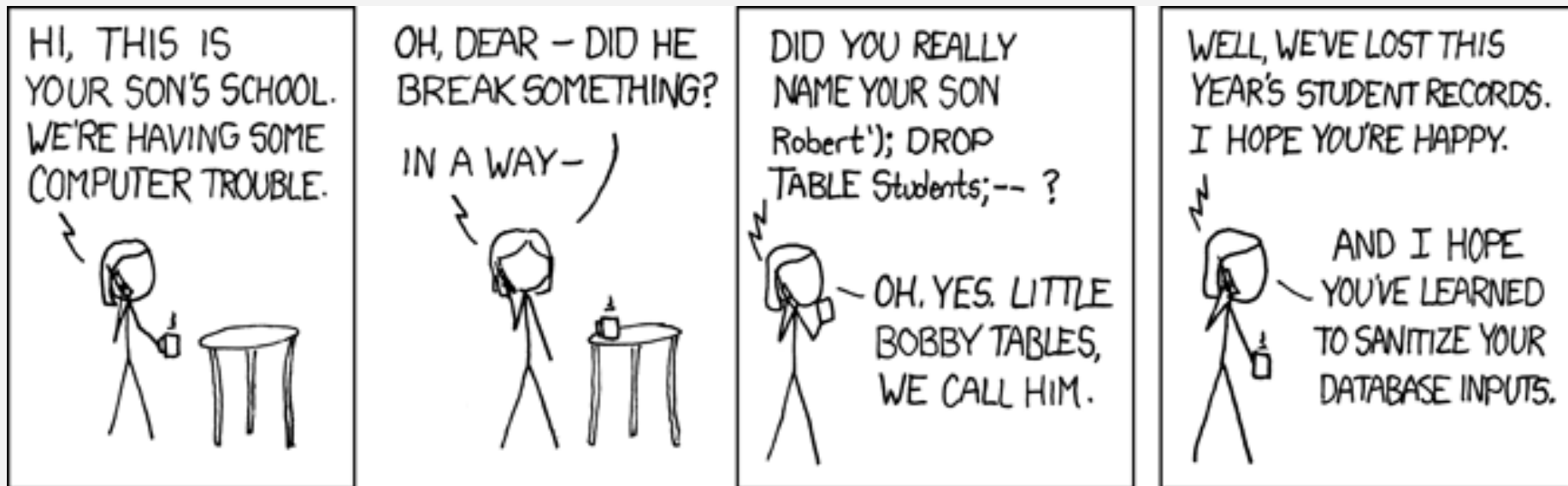


Code Security

SQL Injection

```
txtUserId = getRequestString("UserId");  
txtSQL = "SELECT * FROM Users WHERE UserId = " + txtUserId;
```

User ID: 105; DROP TABLE Suppliers



(Source: <https://xkcd.com/327/>)



4 years

On average, vulnerabilities go undetected for four years before being identified.
Sometimes, even longer than that - Log4j was vulnerable for ~7 years

Static Application Security Testing (SAST)

Analyze source code, bytecode, or binary code to identify security vulnerabilities in the application's codebase.



Whitebox-Testing

- › GitHub Code Analysis
- › SonarQube, Fortify, Checkmarx, and Veracode
- › Semgrep (<https://semgrep.dev/>)
- › Mobile-Security-Framework (MobSF) (<https://github.com/MobSF/Mobile-Security-Framework-MobSF>)

Client-side cross-site scripting
Writing user input directly to the DOM allows for a cross-site scripting vulnerability.

Open Error CWE-79 CWE-116 security

Branch: main Dismiss

```
frontend/src/app/search-result/search-result.component.ts
149   this.io.socket().emit('verifyLocalXssChallenge', queryParam)
150   } // vuln-code-snippet hide-end
151   this.dataSource.filter = queryParam.toLowerCase()
152   this.searchValue = this.sanitizer.bypassSecurityTrustHtml(queryParam) // vuln-code-snippet vuln-line localXssChallenge
```

Cross-site scripting vulnerability due to user-provided value.

[CodeQL](#) [Show paths](#)

```
153   this.gridDataSource.subscribe((result: any) => {
154     if (result.length === 0) {
155       this.emptyState = true

```

Tool	Rule ID	Query
CodeQL	js/xss	View source

Directly writing user input (for example, a URL query parameter) to a webpage without properly sanitizing the input first, allows for a cross-site scripting vulnerability.

[Show more](#)

First detected in commit 9474e68 3 days ago

Create codeql-analysis.yml

frontend/src/app/search-result/search-result.component.ts#L152 on branch main

Verified 9474e68



Tool	Rule ID	Query
CodeQL	js/xss	View source

Directly writing user input (for example, a URL query parameter) to a webpage without properly sanitizing the input first, allows for a cross-site scripting vulnerability.

This kind of vulnerability is also called DOM-based cross-site scripting, to distinguish it from other types of cross-site scripting.

Recommendation

To guard against cross-site scripting, consider using contextual output encoding/escaping before writing user input to the page, or one of the other solutions that are mentioned in the references.

Example

The following example shows part of the page URL being written directly to the document, leaving the website vulnerable to cross-site scripting.

```
function setLanguageOptions() {
  var href = document.location.href;
  deflt = href.substr(href.indexOf("default=")+8);
  document.write("<OPTION value=" + deflt + ">OPTION");
  document.write("<OPTION value=" + deflt + ">OPTION");
}
```

References

- OWASP: [DOM based XSS Prevention Cheat Sheet](#).
- OWASP: [XSS \(Cross Site Scripting\) Prevention Cheat Sheet](#).
- OWASP [DOM Based XSS](#).
- OWASP [Types of Cross-Site Scripting](#).
- Wikipedia: [Cross-site scripting](#).
- Common Weakness Enumeration: [CWE-79](#).
- Common Weakness Enumeration: [CWE-116](#).

[Show less](#)

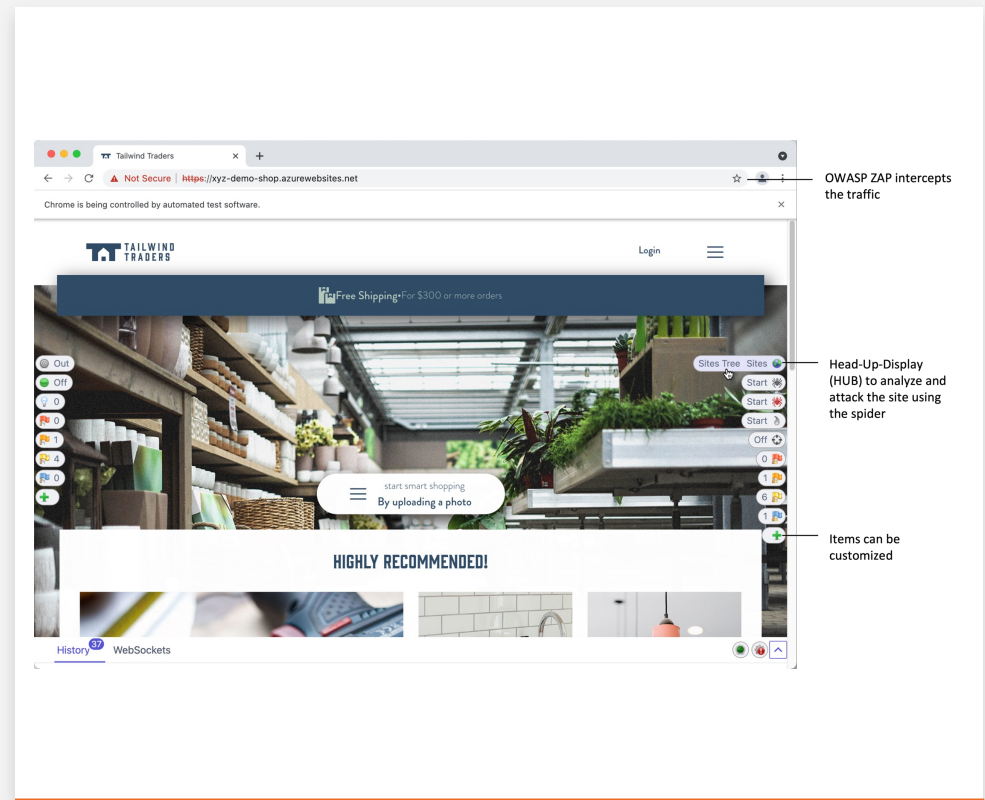
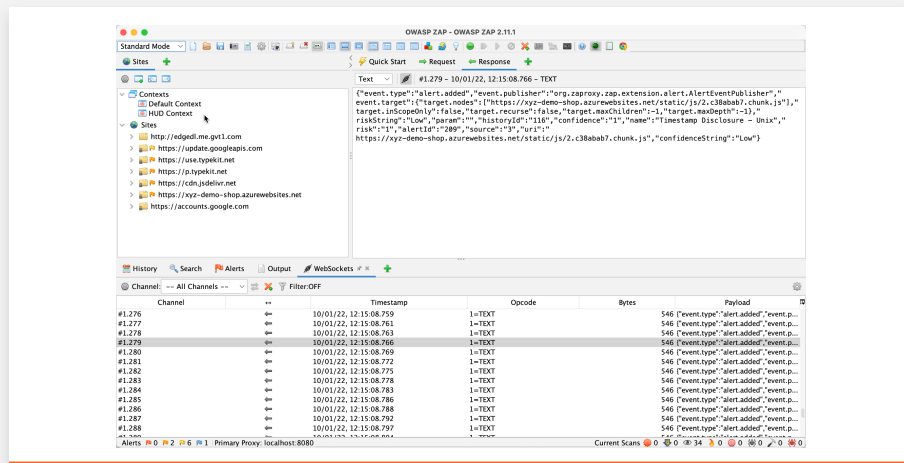
Dynamic Application Security Testing (DAST)

Scans a running application to identify vulnerabilities by sending input requests and analyzing responses.



Blackbox-Testing

- › OWASP ZAP (Zed Attack Proxy, <https://owasp.org/www-project-zap>)
- › Burp Suite von PortSwigger (<https://portswigger.net/burp>)



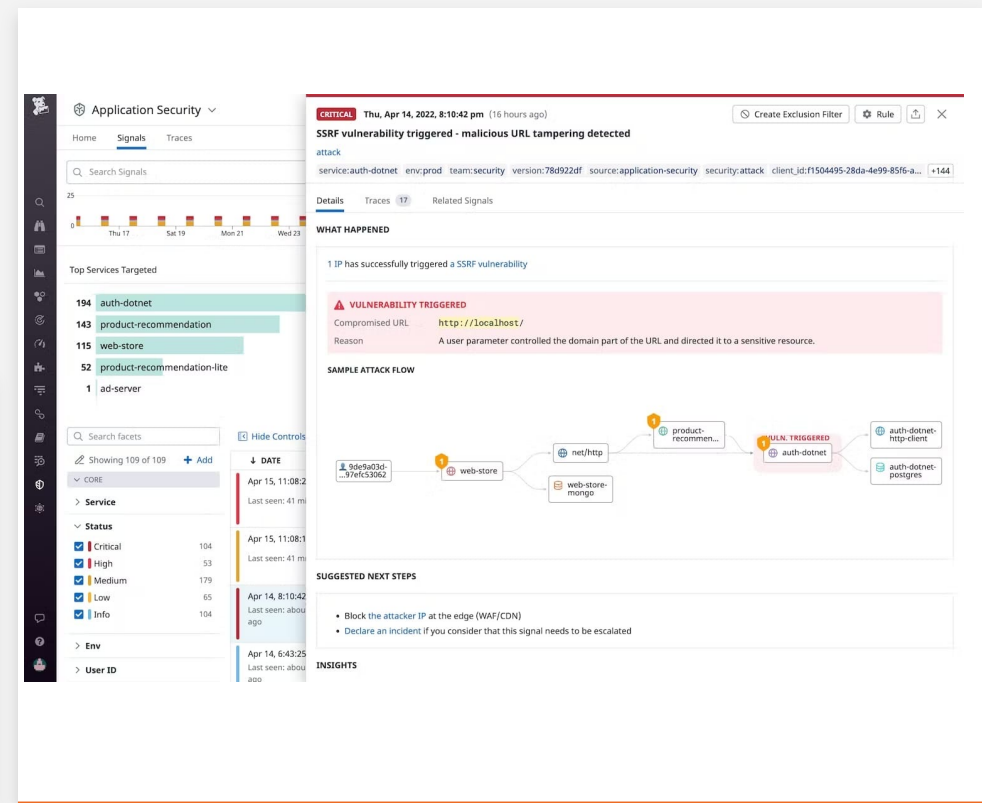
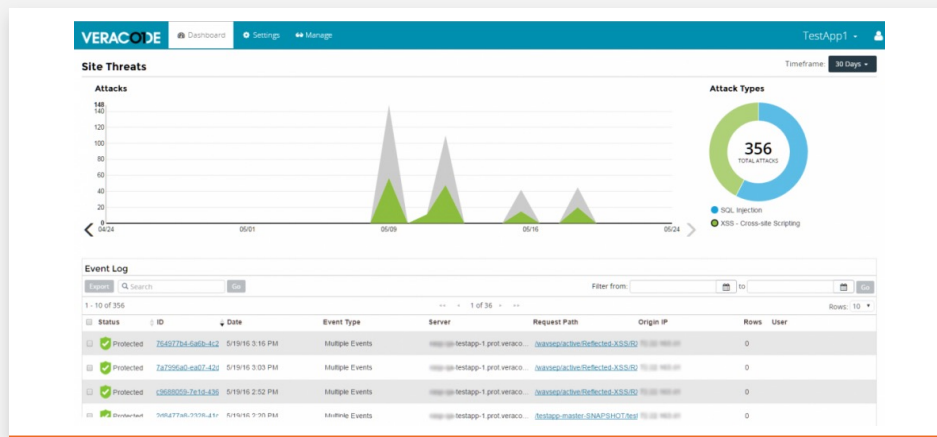
Runtime Application Self-Protection (RASP)

Monitors and protects applications in real-time, detecting and responding to security threats as they occur.



Blackbox-Testing

- › Veracode Runtime Protection
- › F5 Advanced WAF with RASP
- › Datadog Application Security Management



Infrastructure Scanning

Container Vulnerability Analysis (CVA) / Container Security Analysis (CSA)

Open source:

- › Anchore gryp
<https://github.com/anchore/grype/>
- › Clair
<https://quay.github.io/clair/>

Commercial:

- › WhiteSource
<https://www.whitesourcesoftware.com/solution-for-containers/>
- › Aqua
<https://www.aquasec.com/products/container-security/>

```
- name: Anchore Container Scan
  uses: anchore/scan-action@v3.2.0
  with:
    image: ${ env.REGISTRY }}/${ env.IMAGE_NAME }}
    debug: true
```

<https://github.com/wulfland/container-demo/actions/runs/2179243137>



build-and-push-image
failed 3 minutes ago in 22s

Search logs

- Set up job 3s
- Checkout repository 1s
- Log in to the Container registry 0s
- Extract metadata (tags, labels) for Docker 0s
- Build and push Docker image 4s
- Anchore SBOM Action 3s
- Anchore Container Scan 8s**
- Post Build and push Docker image 0s
- Post Log in to the Container registry 0s
- Post Checkout repository 0s
- Complete job 0s

```
1 ▶ Run anchore/scan-action@v3.2.0
12 /usr/bin/chmod +x /home/runner/work/_temp/ca1e5e61-229a-43b7-b70f-c317932ac1c0
13 /home/runner/work/_temp/ca1e5e61-229a-43b7-b70f-c317932ac1c0 -b
14 /home/runner/work/_temp/ca1e5e61-229a-43b7-b70f-c317932ac1c0_grype v0.27.3
15 [info] checking github for release tag='v0.27.3'
16 [info] fetching release script for tag='v0.27.3'
17 anchore/grype info checking GitHub for tag 'v0.27.3'
18 anchore/grype info found version: 0.27.3 for v0.27.3/linux/amd64
19 anchore/grype info installed /home/runner/work/_temp/ca1e5e61-229a-43b7-b70f-
20 c317932ac1c0_grype/grype
21 Analyzing: ghcr.io/wulfland/container-demo
22 Executing: grype -vv -o json --fail-on medium ghcr.io/wulfland/container-demo
23 ▶ grype output...
163 Error: Failed minimum severity level. Found vulnerabilities with level medium or
higher
```

Infrastructure Scanning

Infrastructure policies

Open source:

- › Checkov
<https://www.aquasec.com/products/container-security/>
- › OpenVAS

Commercial:

- › Defender for Cloud
<https://azure.microsoft.com/en-us/services/defender-for-cloud>
- › Azure Policy
<https://docs.microsoft.com/de-de/azure/governance/policy/>

```
- name: Checkov GitHub Action
  uses: bridgecrewio/checkov-action@master
  with:
    directory: ch15_sec/
    output_format: sarif

- name: Upload SARIF file
  uses: github/codeql-action/upload-sarif@v1
  with:
    sarif_file: results.sarif
  if: always()
```

Code scanning

[Add more scanning tools](#)

Latest scan	Branch	Workflow	Lines scanned	Duration	Result
10 minutes ago	main	CodeQL	1.45k / 1.39k ⓘ	5m 26s	21 alerts
Filters ▾ Q tool:checkov is:open branch:main					
✕ Clear current search, filters and sorts					
☐ ✓ 2 Open ✕ 0 Closed Tool ▾ Rule ▾ Branch ▾ Severity ▾ Sort ▾					
☐ Ensure that S3 bucket has a Public Access block ⓘ Error main					
aws.tf:1 • Detected 15 minutes ago by checkov					
☐ Ensure that S3 bucket has cross-region replication enabled ⓘ Error main					
aws.tf:1 • Detected 15 minutes ago by checkov					



CodeQL: a revolutionary semantic code engine



Advanced code analysis engine based on 13 years of research by a 30 person team from Oxford University



Allows you to query your code's logic to find vulnerabilities



Queries can be quickly customized to adapt to your specific threat topology



Community-driven query set powers every project with a world-class security team





Demo: Code analysis (Auto-Fix)



Image thanks to Gregor Biswanger @BFreakout



Hackers in movies
vs

© 2021 Michael Kaufmann @mike_kaufmann

Thank you



Blog : <https://writeabout.net>



Twitter : @mike_kaufmann



GitHub : @wulfland



LinkedIn : <https://www.linkedin.com/in/mikaufmann/>